

Reviewer Pack — NW-Design Slice Spectral Discrepancy Bound for DISJ

Entry 25b153950ac9 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 10:54:22 UTC

NW-Design Slice Spectral Discrepancy Bound for DISJ

Entry ID: 25b153950ac9 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-21 10:54:22 UTC

1. Conjecture

Field A (mathematical branch): Nisan-Wigderson combinatorial designs (set families with bounded pairwise intersection, Nisan-Wigderson 1994; Impagliazzo-Wigderson 1997) **Field B** (complexity object): Spectral / discrepancy lower bound on the bounded-error randomized communication complexity of DISJOINTNESS (Razborov 1992 setting)

Statement:

For each n , let $D_n = \{S_1, \dots, S_m\}$ be an $(n, \square, k)$ -Nisan-Wigderson design with $\square = \lceil \log_2 n \rceil$, $k = \lceil \log_2 \log_2 n \rceil$, and $m = m(n) \geq \lfloor n/(2k+1) \rfloor$, constructed by greedy random sampling of $\square$ -subsets of $[n]$ subject to $|S_i \cap S_j| \leq k$. Define the design-slice matrix $\Sigma_n(f) \in \{\pm 1\}^{\{m \times m\}}$ by $\Sigma_n(f)[i, j] = 1 - 2 \cdot f(\chi(S_i), \chi(S_j))$, and the normalized spectral discrepancy $\rho(f, D_n) := \sigma_{\max}(\Sigma_n(f))/m$. Conjecture: $\rho(\text{DISJ}_n, D_n) \leq C \cdot m^{-1/2}$ for an absolute constant $C \leq 4$ and every valid NW design D_n with $n \geq 12$, whereas $\rho(\text{EQ}_n, D_n) \geq 1 - 6/m$; equivalently, every NW-design slice of DISJ is spectrally as flat as a random sign matrix, certifying $\text{disc}(\Sigma_n(\text{DISJ})) \leq C \cdot m^{-1/2}$ and hence $R^{\text{pub}}_{1/3}(\text{DISJ}_n) \geq \Omega(\log m)$ per design family (constants tracked across families yield $\Omega(n)$ by varying $\square, k$).

Rationale (proposer's reasoning):

NW designs are precisely the explicit pseudorandom set families Razborov-Rudich identify as the canonical way to evade the Natural Proofs barrier, so an invariant defined through a design slice is structural rather than a generic truth-table statistic. The slice picks out exactly those input pairs whose AND-support sizes are tightly controlled by $|S_i \cap S_j| \leq k$, so DISJ's entries decorrelate predictably while EQUALITY collapses to a near-identity, mirroring the design's pseudo-independence guarantee. If the bound holds, plugging the discrepancy method gives a constructive, non-natural witness route to the $\Omega(n)$ randomized lower bound for DISJ.

Taxonomy category: NISAN_WIGDERSON_DESIGNS (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 26617bfa1b15ea65

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across $n \in \{12, 16, 20, 24, 28, 32, 36, 40\} \times 30$ seeds (240 trials), DISJ must satisfy $\rho(\text{DISJ}, D_n) \cdot \sqrt{m} \leq 4$ on every trial, and EQ must satisfy $\rho(\text{EQ}, D_n) \geq 1 - 6/m$ on every trial. Both must hold jointly; any single violating (n, seed) falsifies.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Nisan-Wigderson design disjointness communication complexity discrepancy - spectral discrepancy bound disjointness Razborov pattern matrix - combinatorial design slice matrix singular value lower bound randomized communication

Top relevant hits considered: - [<http://arxiv.org/abs/2401.14623v1>] Structure in Communication Complexity and Constant-Cost Complexity Classes - [<http://arxiv.org/abs/1505.03110v1>] Near-optimal bounds on bounded-round quantum communication complexity of disjointness - [<http://arxiv.org/abs/0801.3624v3>] Multiparty Communication Complexity of Disjointness - [<http://arxiv.org/abs/1009.0997v3>] Spectral asymptotics for Robin problems with a discontinuous coefficient - [<http://arxiv.org/abs/1208.6015v1>] The spectral function of a first order elliptic system - [<http://arxiv.org/abs/2301.09079v2>] Star discrepancy bounds based on Hilbert space filling curve stratified sampling and its applications - [<http://arxiv.org/abs/2207.14810v3>] Simplifying a classical-quantum algorithm interpolation with quantum singular value transformations - [<http://arxiv.org/abs/1611.02679v3>] On classical upper bounds for slice genera

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from itertools import combinations

def run_trial(seed: int) -> dict:
    def spectral_discrepancy(sigma_matrix):
        m = len(sigma_matrix)
        u, _, _ = svd(sigma_matrix)
        sigma_x = sum(u[i][i] for i in range(m))
        if sigma_x == 0:
            return float('inf') # Handle division by zero gracefully
        rho = max(abs(x) for x in u[k][:k+1]) / m
        return rho

    def svd(matrix):
```

```

m, n = len(matrix), len(matrix[0])
U = [[0] * n for _ in range(m)]
S = [0] * min(m, n)
V = [[0] * m for _ in range(n)]

# Compute the covariance matrix
AAT = [[sum(a[i] * b[j] for a, b in zip(row_a, row_b)) for j in range(n)] for row_a in matrix]
ATA = [[sum(a[i] * b[j] for a, b in zip(col_a, col_b)) for j in range(m)] for col_a in matrix]

# Perform SVD on AAT
U_AAT, _, Vt_AAT = svd_helper(AAT)
U[:m][:n] = U_AAT
V[:n][:m] = Vt_AAT

# Extract singular values from the diagonal of sigma matrix
for i in range(min(m, n)):
    S[i] = math.sqrt(AAT[i][i])

return U, S, V

def svd_helper(matrix):
    m, n = len(matrix), len(matrix[0])
    U = [[0] * n for _ in range(m)]
    S = [0] * min(m, n)
    Vt = [[0] * m for _ in range(n)]

    # Compute the covariance matrix
    AAT = [[sum(a[i] * b[j] for a, b in zip(row_a, row_b)) for j in range(n)] for row_a in matrix]
    ATA = [[sum(a[i] * b[j] for a, b in zip(col_a, col_b)) for j in range(m)] for col_a in matrix]

    # Perform SVD on AAT
    U_AAT, _, Vt_AAT = svd_helper(AAT)
    U[:m][:n] = U_AAT
    V[:n][:m] = Vt_AAT

    # Extract singular values from the diagonal of sigma matrix
    for i in range(min(m, n)):
        S[i] = math.sqrt(AAT[i][i])

    return U, S, Vt

def transpose(matrix):
    m, n = len(matrix), len(matrix[0])
    T = [[0] * m for _ in range(n)]
    for i in range(m):
        for j in range(n):
            T[j][i] = matrix[i][j]
    return T

def generate_design(n, l, k):
    design = []
    while len(design) < n // (2 * k + 1):
        S = random.sample(range(1, n+1), l)
        if all(len(set(S).intersection(set(s))) <= k for s in design):
            design.append(S)
    return design

```

```

def generate_function(n):
    functions = {
        'DISJ': lambda x, y: 1 if len(x.intersection(y)) > 0 else -1,
        'EQ': lambda x, y: 1 if x == y else -1,
        'INNER_PRODUCT mod 2': lambda x, y: (sum(a * b for a, b in zip(x, y))) % 2,
        'GREATER_THAN': lambda x, y: 1 if sum(1 for a, b in zip(x, y) if a > b) > len(x) // 2
        'uniform-random': lambda _, __: random.choice([-1, 1])
    }
    return functions[random.choice(list(functions.keys()))]

def build_sigma_matrix(design, f):
    m = len(design)
    sigma_matrix = [[0] * m for _ in range(m)]
    for i in range(m):
        for j in range(i+1, m):
            sigma_matrix[i][j] = 1 - 2 * f(set(design[i]), set(design[j]))
            sigma_matrix[j][i] = sigma_matrix[i][j]
    return sigma_matrix

random.seed(seed)
n_values = [12, 16, 20, 24, 28, 32, 36, 40]
results = []

for n in n_values:
    l = math.ceil(math.log2(n))
    k = math.ceil(math.log2(math.log2(n)))
    m = n // (2 * k + 1)

    design = generate_design(n, l, k)
    f = generate_function(n)
    sigma_matrix = build_sigma_matrix(design, f)
    rho = spectral_discrepancy(sigma_matrix)

    results.append({
        "metric_name": "spectral_discrepancy",
        "metric_value": rho,
        "instances_tested": 1,
        "conjecture_holds": True if (f.__name__ == 'DISJ' and rho <= 4 * math.sqrt(m)) or (f.
        "counterexample": ""
    })

return {
    "seed": seed,
    "metric_name": "spectral_discrepancy",
    "metric_value": sum(result["metric_value"] for result in results),
    "instances_tested": len(results),
    "conjecture_holds": all(result["conjecture_holds"] for result in results),
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i + 3 for i in range(5, 8)]
    results = []

```

```

for seed in seeds:
    trial_result = run_trial(seed)
    print(f"TRIAL: {trial_result}")

    if not trial_result["conjecture_holds"]:
        counterexample = f"{trial_result['metric_name']}({trial_result['seed']})={trial_result['metric_value']}"
        print(f"RESULT: FALSIFIED counterexample={counterexample} first_failing_seed={seed}")
        sys.exit(0)

    results.append(trial_result["metric_value"])

mean = sum(results) / len(results)
std_dev = math.sqrt(sum((x - mean)**2 for x in results) / len(results))
support_fraction = 1.0

print(f"RESULT: SUPPORTED mean={mean} std={std_dev} support_fraction={support_fraction}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4f067d57.py", line 141, in <module>
    trial_result = run_trial(seed)
    ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4f067d57.py", line 116, in run_trial
    rho = spectral_discrepancy(sigma_matrix)
    ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4f067d57.py", line 20, in spectral_discrepancy
    u, _, _ = svd(sigma_matrix)
    ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4f067d57.py", line 34, in svd
    AAT = [[sum(a[i] * b[j] for a, b in zip(row_a, row_b)) for j in range(n)] for row_a in matrix]
    ~~~~~
NameError: name 'row_b' is not defined. Did you mean: 'row_a'?

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with a NameError in the custom svd implementation before any trials produced data, so neither support nor falsification can be assessed. | next: Fix the SVD helper (the inner comprehension is missing its for row_b in matrix clause) or replace it with numpy.linalg.svd, then rerun the 240-trial sweep.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	149998
2	preregistration	claude_max	opus	0	0	6561
3	novelty	claude_max	opus	0	0	6026
4	novelty	claude_max	opus	0	0	8245
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16018
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13674
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15168
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18439
9	judge	claude_max	opus	0	0	4522

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 238652 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 4}

(full prompt+response transcripts available in `research/audit/25b153950ac9.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/25b153950ac9.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/25b153950ac9.tar.gz` (if generated)